Kajaani University of Applied Sciences

Intelligent Systems Engineering

# VisualQC

Development and evaluation of a camera-based visual quality inspection prototype

Technical project report

Antti Koljonen

25 October 2026

Project version 0.1

## Abstract

VisualQC is a camera-based visual quality inspection prototype designed to classify paper test pieces with printed circular marks. The circles represent required manufactured features. The project combines classical image processing with a pretrained ResNet18 neural network adapted using PyTorch. It covers image capture, human labeling, data preparation, geometric paper detection, perspective cropping, neural-network training, GPU inference, and a real-time inspection interface. The system reports PASS, FAIL, or NO COMPONENT at the operational level and displays disagreement between the OpenCV and neural-network methods.

The first documented training experiment produced a 76.92% test accuracy (10 correct predictions among 13 images), but that holdout was small. Inspection of the supplied development archive revealed 240 labeled files but only 157 distinct file contents, so duplicate leakage is a substantial concern for image-level splits. Live screenshots also demonstrate both correct detection and errors, including false acceptance of three-circle sheets. The main conclusion is that the end-to-end proof of concept operates, but reliable independent evaluation and dataset controls are needed before the model can be considered effective for quality assurance.

Keywords: machine vision, PyTorch, ResNet18, transfer learning, OpenCV, manufacturing inspection, defect detection, CUDA

## Contents

1 Introduction

2 Requirements and technologies

3 System implementation

4 Dataset and model training

5 Results and observations

6 Limitations and improvement plan

7 Conclusions

References

# 1 Introduction

Visual inspection is common in manufacturing quality assurance. Simple geometric defects can often be detected with predefined image-processing rules, while variable appearance may motivate machine learning. The project investigates both approaches on an accessible physical test bench: a Logitech C922 webcam observes light paper sheets placed against a dark surface. Black circular marks simulate holes or other required features. A four-mark piece is considered acceptable and a piece with fewer than four marks is defective.

The objective was to build a working prototype and gain practical experience with the full AI engineering workflow, rather than to prove superiority of a neural network over conventional machine vision. The planned output is a demonstrable portfolio project with reproducible software and honest evaluation.

# 2 Requirements and technologies

| Component | Implementation |
|---|---|
| Camera | Logitech C922 USB webcam; MJPEG requested at 1280 × 720 and 60 FPS |
| Computer | AMD Ryzen 7 7800X3D, 64 GB RAM, NVIDIA RTX 4080 SUPER 16 GB |
| Development | Windows, Python, Visual Studio Code, virtual environment |
| Vision | OpenCV and NumPy for thresholding, contours, paper detection and warping |
| Machine learning | PyTorch and torchvision pretrained ResNet18, two output classes |
| Training support | scikit-learn splitting, training-only image augmentation, CUDA when available |

The camera was requested to run at 60 FPS, but several screenshots show processed throughput around 10–11 FPS. Network-free runtime benchmarking under controlled conditions has not been documented; the displayed FPS is not the same as camera hardware delivery rate.

# 3 System implementation

## 3.1 Image acquisition and labeling

The capture program acquires webcam frames, supports focus control, and saves timestamped images. The rule-based inspection program thresholds the image, finds contours and evaluates contour area and circularity. A manual keyboard labeling workflow then copies images to the good, defective or inconclusive directories. This human review separates training labels from the automated detector’s potentially incorrect predictions.

## 3.2 Paper localization and standardization

The later pipeline detects a bright, approximately rectangular sheet on a darker background. A grayscale brightness threshold and morphological closing produce candidate components; contour area and rectangularity tests select a paper region. The fitted rotated rectangle is converted into four corners and warped to a 224 × 224 crop. Live inference is skipped if no suitable component is located, producing NO COMPONENT rather than forcing a two-class model to classify the empty scene.

The paper detector is sensitive to material brightness, surface contrast, lighting and geometric assumptions. The fitted rectangle is an approximation, particularly if the sheet is strongly distorted, curled, or partially outside the camera view.

## 3.3 Parallel inspection methods

OpenCV estimates the number of circular marks in the component crop. The neural network separately predicts GOOD or DEFECTIVE using normalized RGB image data. The interface maps the neural-network classes to PASS/FAIL and compares them with the rule-based decision. A warning appears when the methods disagree. Confidence is computed from softmax scores; it has not been calibrated as a probability of correctness.

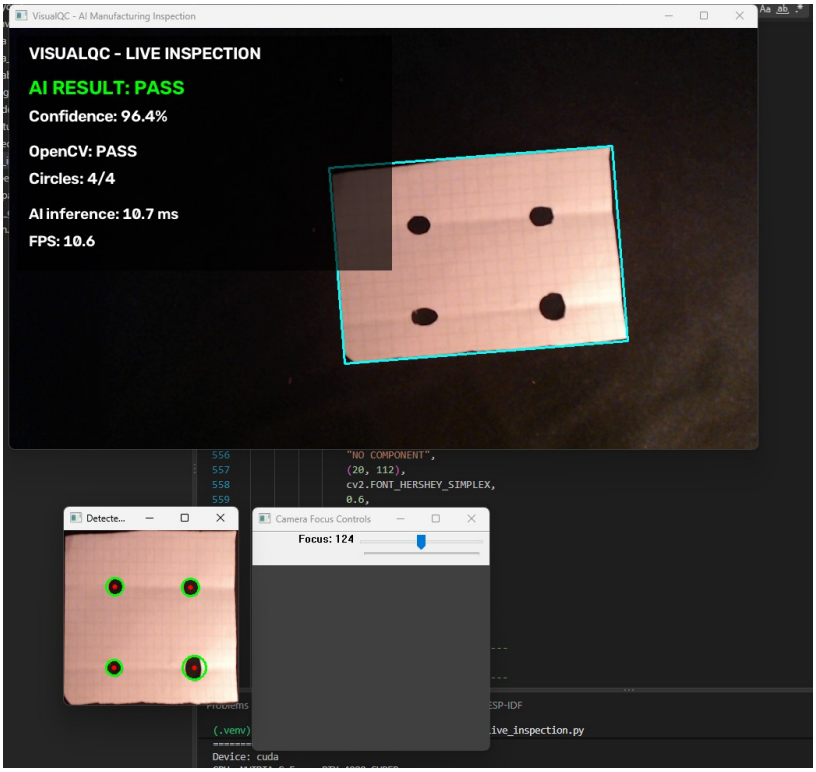



*Figure 1. Cropped-pipeline live inspection with a correctly accepted four-circle sheet.*

# 4 Dataset and model training

## 4.1 Dataset provenance and integrity

The supplied ZIP contains 240 labeled image files: 161 in good, 77 in defective and 2 in inconclusive. A byte-level SHA-256 audit found 157 unique image contents and 83 duplicate copies. The cropped dataset directory contains 151 good and 67 defective image files. These are file counts, not counts of independent physical parts or capture sessions.

The script uses a stratified 70/15/15 image-level train/validation/test split with a fixed random seed. It does not keep duplicates or near-identical frames from one capture session in the same split. Consequently, validation and test metrics from the current workflow cannot establish generalization to unseen physical components. Future training must deduplicate and use group-based splitting with session or sheet identifiers.

| Dataset snapshot | Good | Defective | Inconclusive | Total |
|---|---|---|---|---|
| Raw labeled files in supplied ZIP | 161 | 77 | 2 | 240 |
| Prepared cropped files | 151 | 67 | — | 218 |
| Unique byte contents in raw labels | — | — | — | 157 |

## 4.2 Model and training procedure

The project uses ResNet18 initialized with ImageNet pretrained weights. In the latest supplied train.py, pretrained layers are frozen except for the final convolutional block, layer4, and the replacement two-unit classification layer. Training uses weighted cross-entropy to counter class imbalance, Adam optimization with different learning rates for layer4 (0.00001) and the classifier (0.0001), batch size 16 and 30 epochs. Random rotation and brightness/contrast jitter are used only on the training subset. Evaluation resizing and ImageNet normalization are mirrored during live inference.

The best model is selected using validation accuracy and saved as a PyTorch checkpoint with class names and a test-accuracy field. The newest cropped checkpoint exists in the supplied archive. Its numerical validation and test metrics were not supplied in a verified training log, so they are not asserted here.

# 5 Results and observations

## 5.1 Initial experimental accuracy

An earlier, smaller ResNet18 run reported 57 training images, 12 validation images and 13 test images. The final training accuracy was 71.93%, the best validation accuracy 66.67%, and test accuracy 76.92% (10/13). These figures describe the initial model only. They must not be used as an independent benchmark of the cropped model or operational inspection system.

### 5.2 Live behavior and failure cases

The screenshots demonstrate a correct four-circle PASS using both methods, and correct FAIL decisions on an empty paper and a three-circle sheet. Other frames show errors: before cropping, the neural network accepted a three-circle sheet with 82.3% confidence while OpenCV counted three circles; another image with four circles arranged in a row was classified FAIL by the network while OpenCV counted four. The observed discrepancy suggests sensitivity to training patterns and configuration, but the cause requires controlled tests.

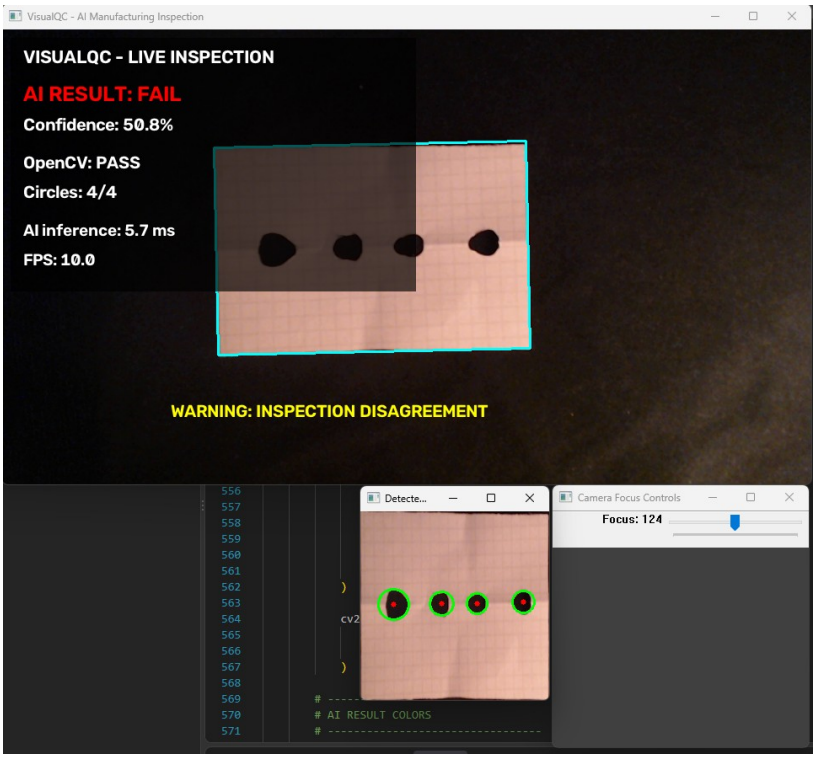



*Figure 2. Example of disagreement on a four-circle pattern in a different arrangement.*

Screenshots of empty backgrounds also showed predictions fluctuating near 50% before the paper-presence gate was introduced. Such behavior is unsurprising for a binary classifier presented with an out-of-distribution image. The gate prevents those frames from being treated as inspected components, although it is itself heuristic.

### 5.3 Performance

Visible screenshots of live inspection show approximately 3–13 ms AI inference times and 10–11 processed frames per second. These are observed on-screen values, not results from a controlled latency benchmark. The program includes resizing, paper localization, UI rendering and camera acquisition in addition to neural-network inference. A systematic benchmark should record each component separately and report medians and percentiles.

## 6 Limitations and improvement plan

| Priority | Issue and proposed implementation | Verification criterion |
|---|---|---|

| | | |
|---|---|---|
| 1 — critical | Remove exact duplicates, record sheet and session IDs, and use group-stratified train/validation/test splits. Preserve a separate untouched test session. | No duplicate hashes or shared session groups across splits. |
| 2 — critical | Measure confusion matrices and per-class recall, especially false PASS on a defective sample; save predictions and labels in a CSV. | Report defective recall, false acceptance rate and sample counts. |
| 3 — high | Balance defect examples across missing-circle positions and zero-, one-, two- and three-circle cases. Include new physical sheets. | Comparable recall across defect subtypes on independent sheets. |
| 4 — high | Verify four expected feature positions and plausible sizes rather than circle count alone; reject unusual arrangements. | Four circles in the wrong locations no longer pass geometrically. |
| 5 — high | Replace fixed light-on-dark thresholds with calibrated illumination, quadrilateral validation, and image-quality checks. | Stable component detection across specified lighting and rotation ranges. |
| 6 — medium | Calibrate neural-network scores, introduce a review-required state for disagreement or uncertain inputs, and avoid treating softmax as certainty. | Reduced high-confidence false acceptances on held-out data. |
| 7 — medium | Measure camera acquisition, preprocessing, GPU inference and rendering separately; reduce frame resolution or process every Nth frame as necessary. | Documented throughput, end-to-end latency and CPU/GPU utilization. |
| 8 — medium | Add deterministic preprocessing tests, model/version metadata, dataset hashes, and reproducible run configuration. | A clean environment can reproduce a documented experiment. |

A central design tradeoff is that a four-circle geometric pattern may be easier to inspect with deterministic machine vision than with a general image classifier. The project should compare both approaches fairly. A neural network adds value only if it improves robustness to specified variability or extends inspection to defects that geometric rules cannot represent simply.

## 7 Conclusions

VisualQC successfully links hardware capture, manually labeled images, preprocessing, pretrained neural-network adaptation and a responsive live inspection interface. The project demonstrates a functional applied AI development workflow, including practical debugging of camera focus, contour hierarchy, input standardization and classification disagreement. It is a suitable educational proof of concept and engineering portfolio item.

The current evidence does not support a production reliability claim. The major next milestone is not a larger reported accuracy percentage: it is a duplicate-free, session-independent evaluation that measures false acceptance of defective components. Once that baseline is established, model changes and additional data can be tested objectively.

## References

PyTorch. Transfer Learning for Computer Vision Tutorial. https://docs.pytorch.org/tutorials/beginner/transfer_learning_tutorial.html

Torchvision. ResNet18 model documentation. https://docs.pytorch.org/vision/stable/models/generated/torchvision.models.resnet18.html

OpenCV. Image thresholding and contour features documentation. https://docs.opencv.org/

VisualQC version 0.1 supplied source-code archive and screenshots, 25 October 2026. Project implementation and observations supplied by Antti Koljonen.